

BUILT — Data protection: root cause · safeguard · recovery (v0.29.0)

The 'truncation' was a sandbox read-cache artifact (your files were never damaged). We still built real protection: atomic writes, a snapshot vault, damage detection, and byte-for-byte recovery.

1 · ROOT CAUSE (reproduced)

guest writes -> guest reads

the Linux sandbox reading its own writes is ALWAYS coherent (47,843 bytes out = in, even short->long re-edits).

= NOT the problem

host writes -> guest reads

after the editor (host) rewrites a file, the sandbox page cache serves a STALE shorter length until it revalidates. Host file stays intact.

= the artifact (read-side only)

2 · SAFEGUARD (engine/safeguard.py) — atomic writes · snapshot vault · verify · recover



verify classifies: ■ OK ■ TRUNCATED ■ CORRUPTED ■ EMPTY ■ MISSING ■ MODIFIED

Heavy viewer.db: SQLite integrity_check always; consistent online-backup copy only on demand (/withdb). Corpus (E:\) is read-only source, not snapshotted. Snapshots are additive (R6); main index never modified (R1).

3 · TESTED TO A STRANGLEHOLD

19 / 19

pillar tests pass

11 / 11

truncation + recovery tests pass

36 / 38

mutants killed (95%) — 2 equivalent

Damage deliberately injected & recovered byte-for-byte:

- ✓ last line (light)
- ✓ 50% (medium)
- ✓ 10 bytes (hard)
- ✓ 0 bytes (empty)
- ✓ partial UTF-8
- ✓ byte-flip corrupt
- ✓ deleted (missing)
- ✓ multi-file
- ✓ corrupted vault relic
- ✓ corrupted DB header

Mutation testing — 2 rounds:

Round 1 · engine logic (core_pillars)

26 faults injected -> 25 killed (96%)

Round 2 · safeguard itself

12 faults injected -> 11 killed (92%)

Round 2 mutates the protection layer (blind-to-missing, trunc->ok, skip-write, hash-blind, dbcheck-always-ok ...) and confirms the truncation suite catches them — the safeguard guards are themselves guarded. Survivors are provably equivalent mutants.